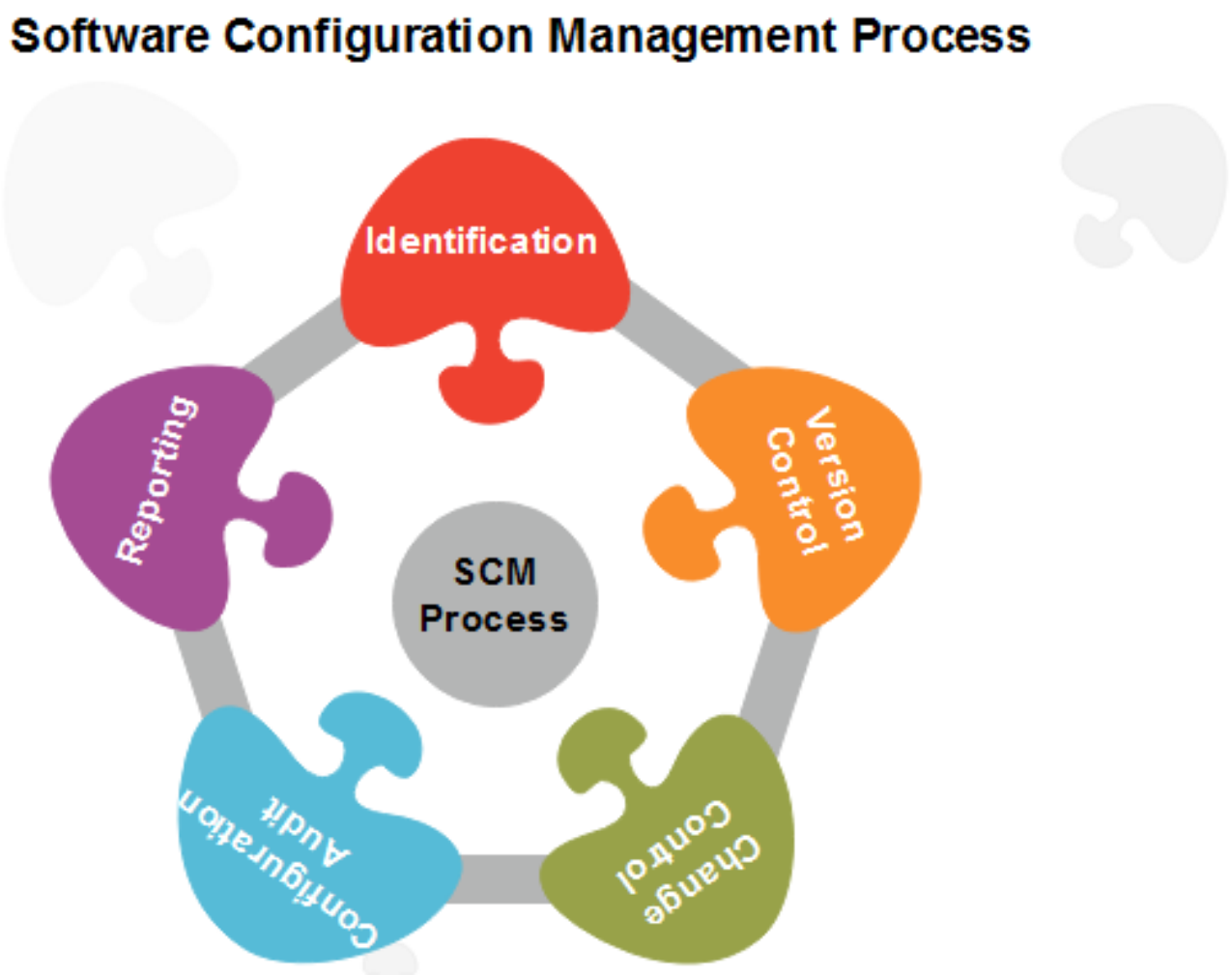
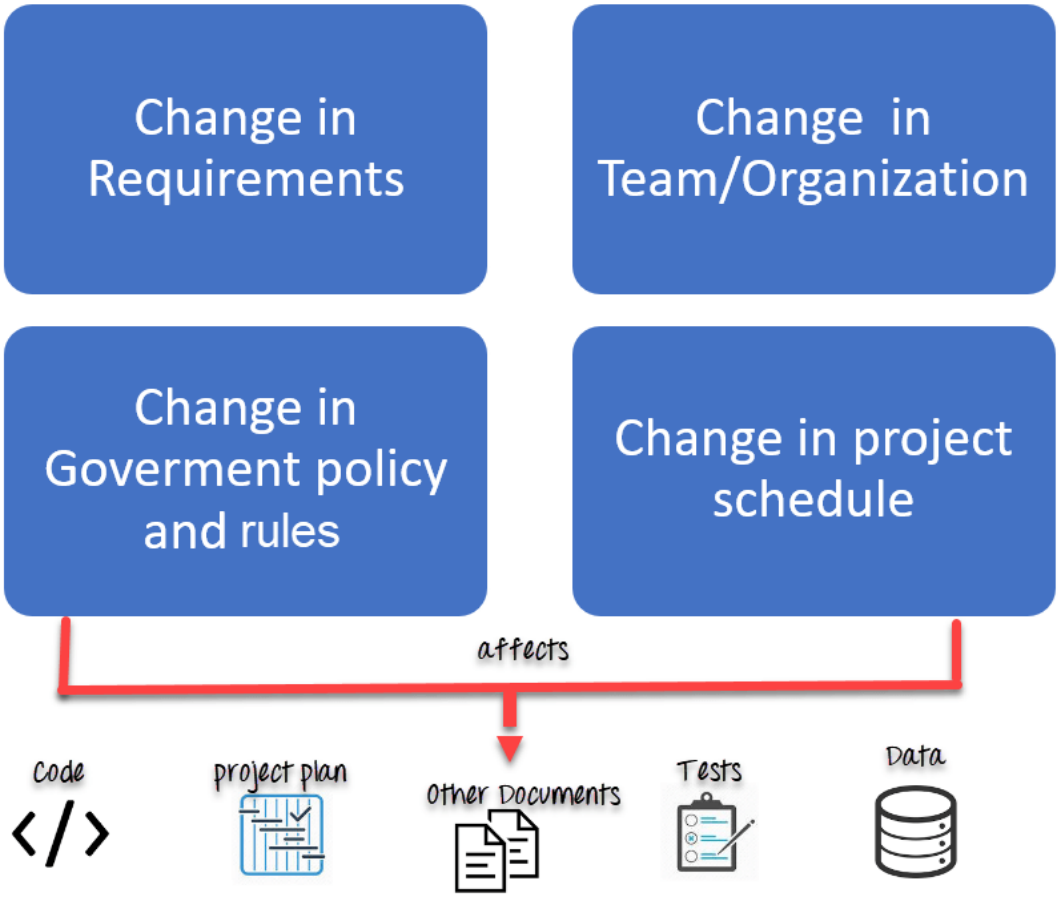
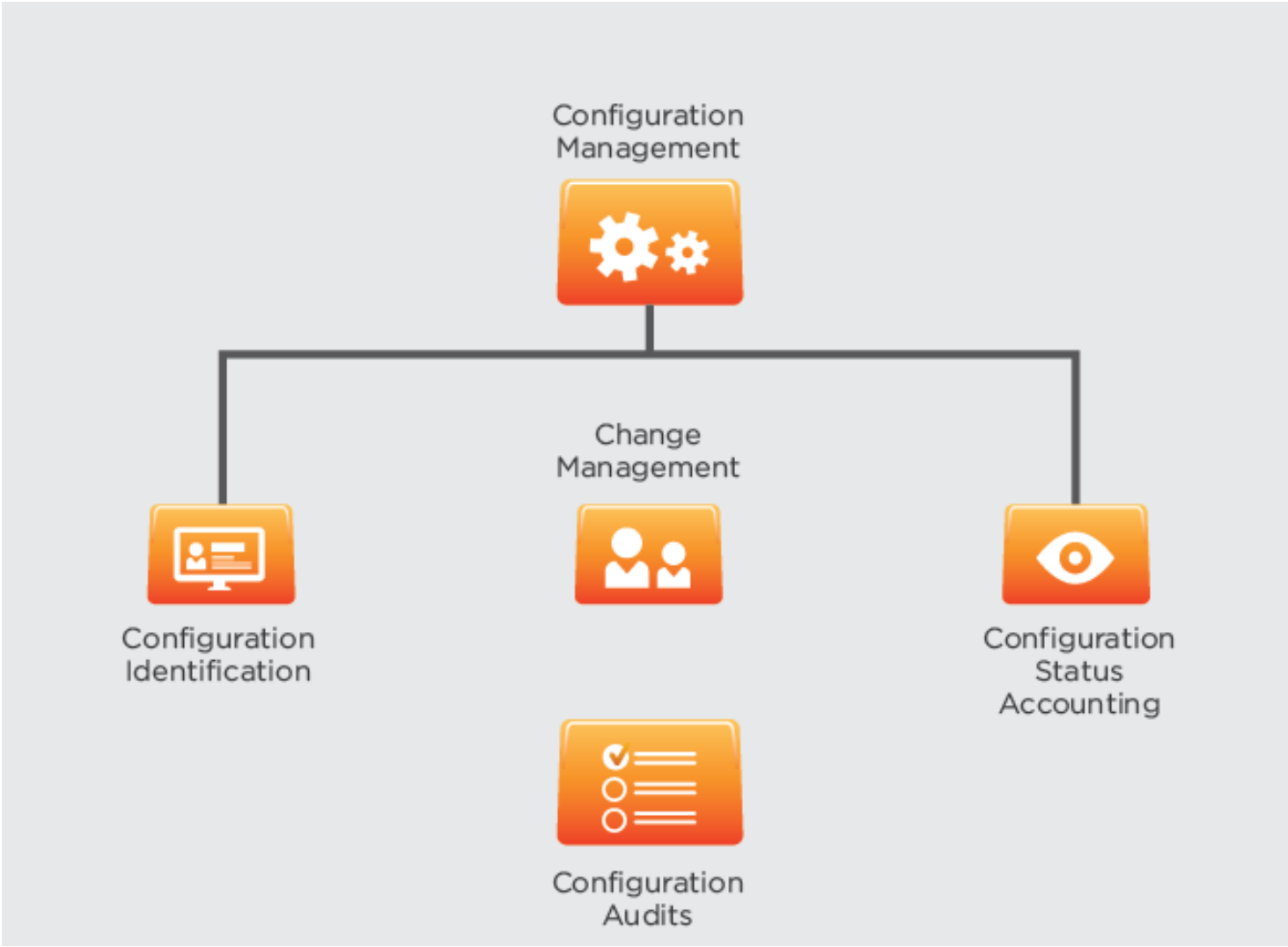
Software configuration management



- There are multiple people working on software which is continually updating
- It may be a case where multiple version, branches, authors are involved in a software config project, and the team is geographically distributed and works concurrently
- Changes in user requirement, policy, budget, schedule need to be accommodated.
- Software should able to run on various machines and Operating Systems
- Helps to develop coordination among stakeholders
- SCM process is also beneficial to control the costs involved in making changes to a system
-


Change in Requirements
Change in Team/Organization
Change in Goverment policy and rules
Change in project schedule
affects
Code
project plan
Other Documents
Tests
Data


Configuration Management
Change Management
Configuration Identification
Configuration Status Accounting
Configuration Audits

If you don't know about
the problem,
Life can be very peaceful

Software Reuse

What is software reuse?

Software reuse is the process of implementing or updating software systems using existing software assets.

- The systematic development of reusable components
- The systematic reuse of these components as building blocks to create new system

The advantages of reuse

- Increase software productivity
- Shorten software development time
- Improve software system interoperability
- Develop software with fewer people
- Move personnel more easily from project to project
- Reduce software development and maintenance costs
- Produce more standardized software
- Produce better quality software and provide a powerful competitive advantage

What is reusable?

- Application system
- Subsystem
- Component
- Module
- Object
- Function or Procedure

Stages of reuse development

- Identify domain
- Identify and classify reusable abstractions
- Identify design/programming language constructs that support reuse
- Study and formulate language reuse guidelines
- Study and formulate domain reuse guidelines
- Reuse assessment—assess components based on the guidelines
- Reuse improvement—modify and improve these components.

The problem in software reuse

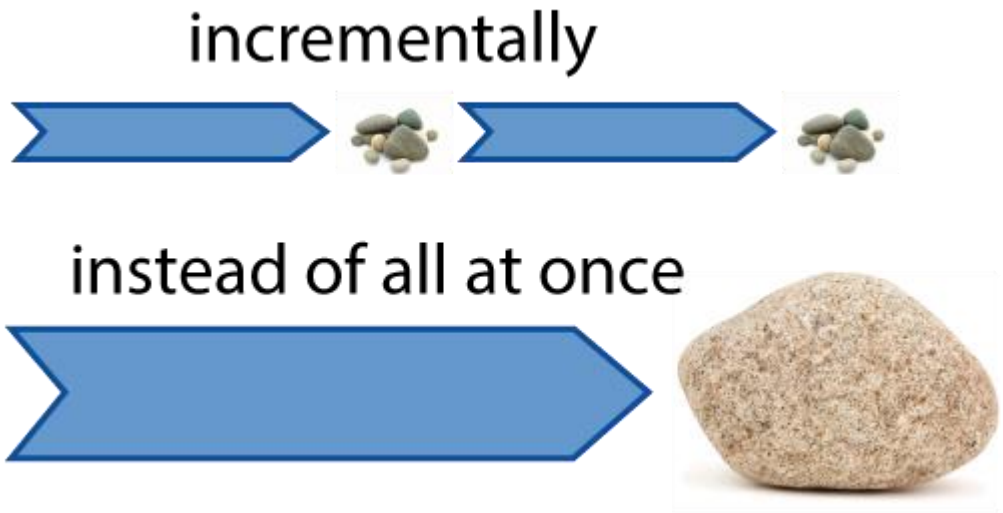
- The principles, methods, and skills required to develop reusable software cannot be learned effectively by generalities and platitudes.
- To succeed in-the-large, reuse efforts must address both technical and non-technical issues.
- It's easier and more cost effective to develop and evolve networked applications by basing them on reusable distributed object computing middleware, which is software that resides between applications and the underlying operating systems, network protocol stacks, and hardware.

Component--based development

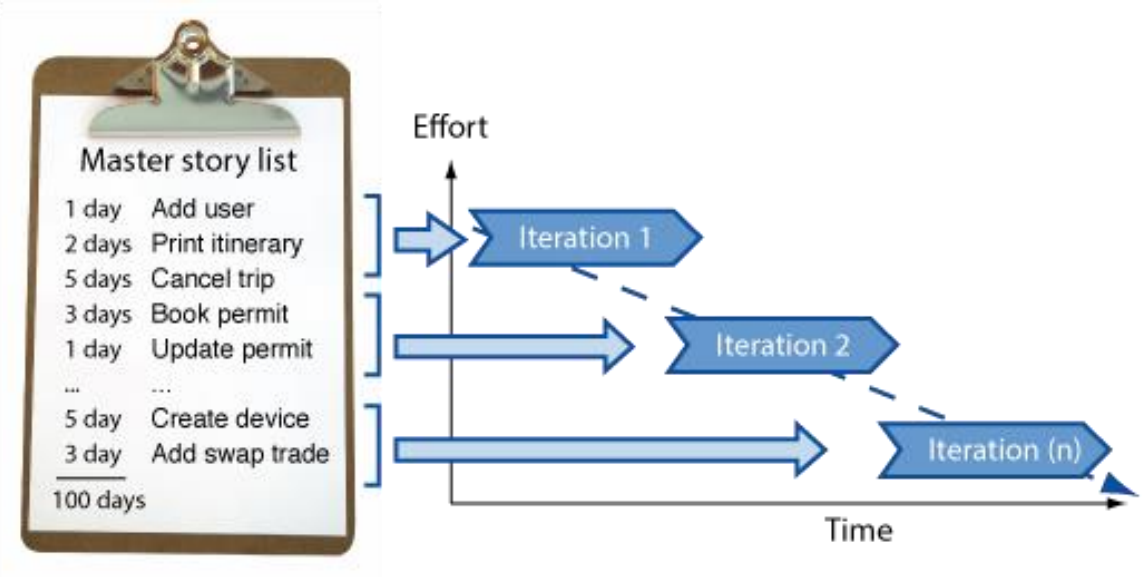
Component--based development

- Component--based software engineering (CBSE) is an approach to software development that relies on software reuse.
- It emerged from the failure of object--oriented development to support effective reuse. Single object classes are too detailed and specific.
- Components are more abstract than object classes and can be considered to be stand--alone service providers.
- Independent components specified by their interfaces.
- Component standards to facilitate component integration.
- Middleware that provides support for component inter--operability.
- A development process that is geared to reuse.

- **What is Agile?**
- Agile is a time boxed, iterative approach to software delivery that builds software incrementally from the start of the project, instead of trying to deliver it all at once near the end.



- It works by breaking projects down into little bits of user functionality called user stories, prioritizing them, and then continuously delivering them in short two week cycles called iterations.



AGILE BASICS

AGENDA

- Agile origins – Manifesto
- Scrum
- Kanban
- eXtreme Programming (XP)

AGILE MANIFESTO (2001)

www.agilemanifesto.org

We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over **processes and tools**

Working software over **comprehensive documentation**

Customer collaboration over **contract negotiation**

Responding to change over **following a plan**

That is, while there is value in the items on
the right, we value the items on the left more.

http://agilemanifesto.org/

PRINCIPLES BEHIND THE AGILE MANIFESTO

1. Our highest priority is to **satisfy the customer** through early and **continuous delivery of valuable software**.

2. **Welcome changing requirements**, even late in development. Agile processes **harness change** for the customer's competitive advantage.

3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

4. Business people and developers must **work together** daily throughout the project.

5. Build projects around **motivated individuals**. Give them the environment and support they need, and trust them to get the job done.

6. The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**.

Principles behind the Agile Manifesto

7. **Working software** is the primary **measure** of progress.

8. Agile processes promote **sustainable development.** The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

9. Continuous attention to **technical excellence** and **good design** enhances agility.

10. **Simplicity**--the art of maximizing the amount of work not done--is essential.

11. The best architectures, requirements, and designs emerge from **self-organizing teams.**

12. At regular intervals, the **team reflects** on how to become more effective, then tunes and adjusts its behavior accordingly.

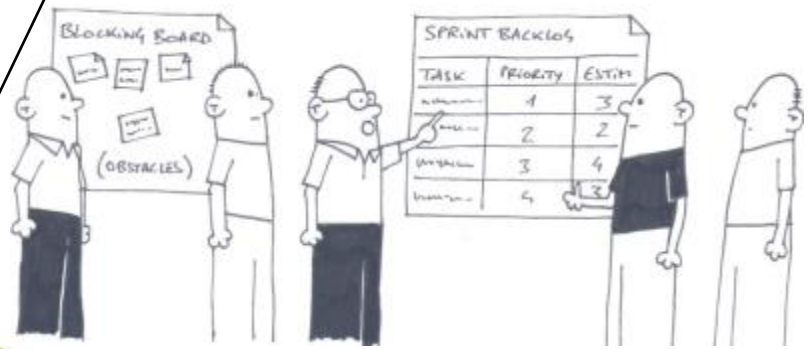
SCRUM OVERVIEW

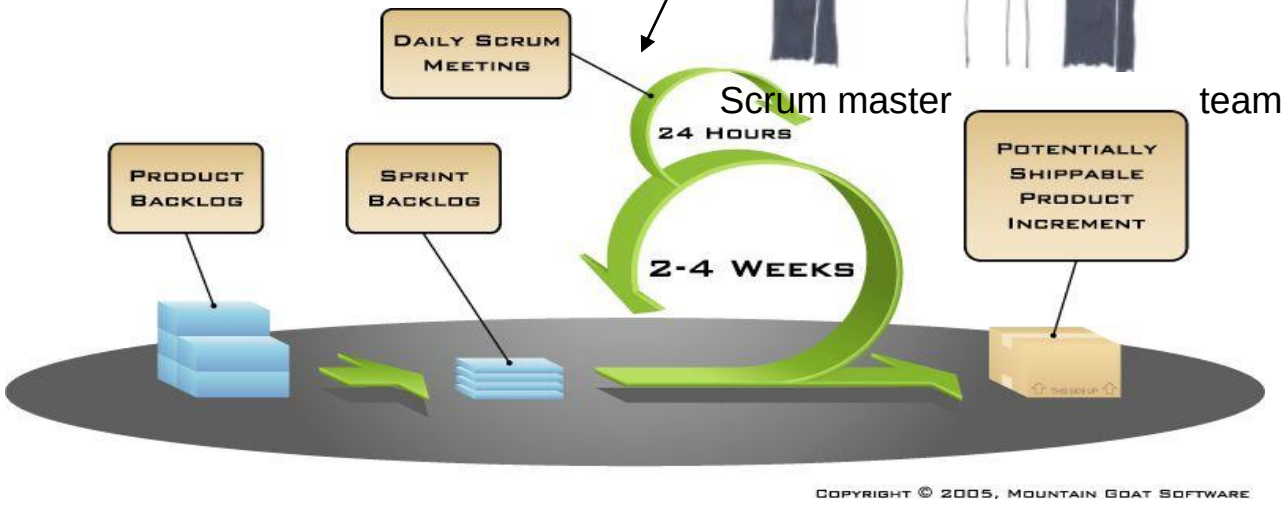
- Scrum is an Agile process;
- Used to manage complex projects since 1990;
- Delivers business functionality in 30 days;
- Business sets the priorities;
- Teams self-organize to determine the best way to deliver the highest priority features.
- Scalable to distributed, large, and long projects;
- Extremely simple but very hard!

SCRUM - FRAMEWORK

Timeboxing!

- Sprint planning – “definition of Done”
- Sprint review – “the demo”
- Sprint retrospective
- Daily scrum meeting





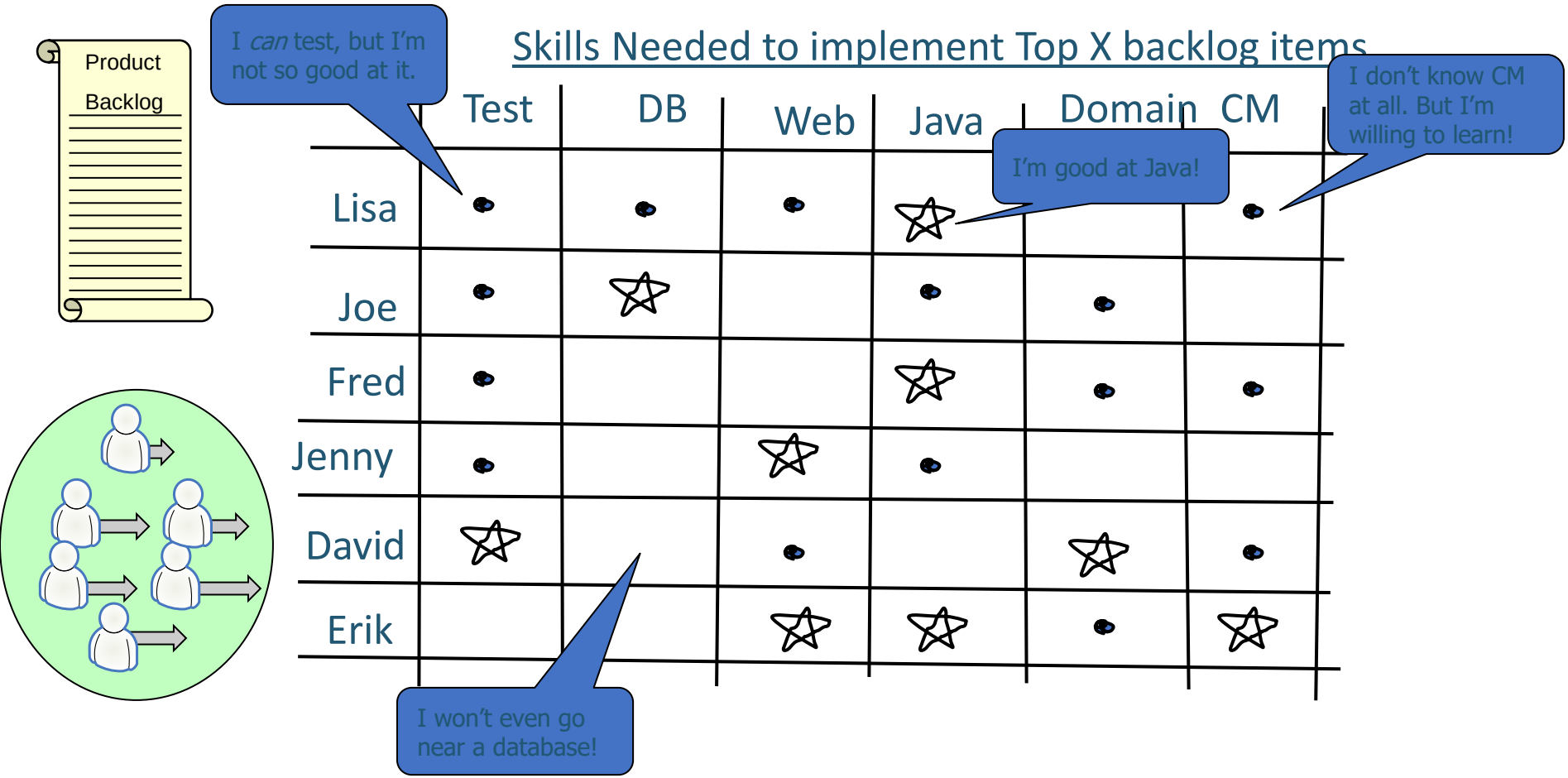
Scrum master

team

Product owner

Cross functional team

Doesn't mean everyone has to know everything



Henrik Kniberg

TEAM

- Seven (plus/minus two) members
- Is cross-functional (Skills in testing, coding, architecture etc.)
- Selects the Sprint goal and specifies work results
- Has the right to do everything within the boundaries of the project guidelines to reach the Sprint goal
- Organizes itself and its work
- Demos work results to the Product Owner.

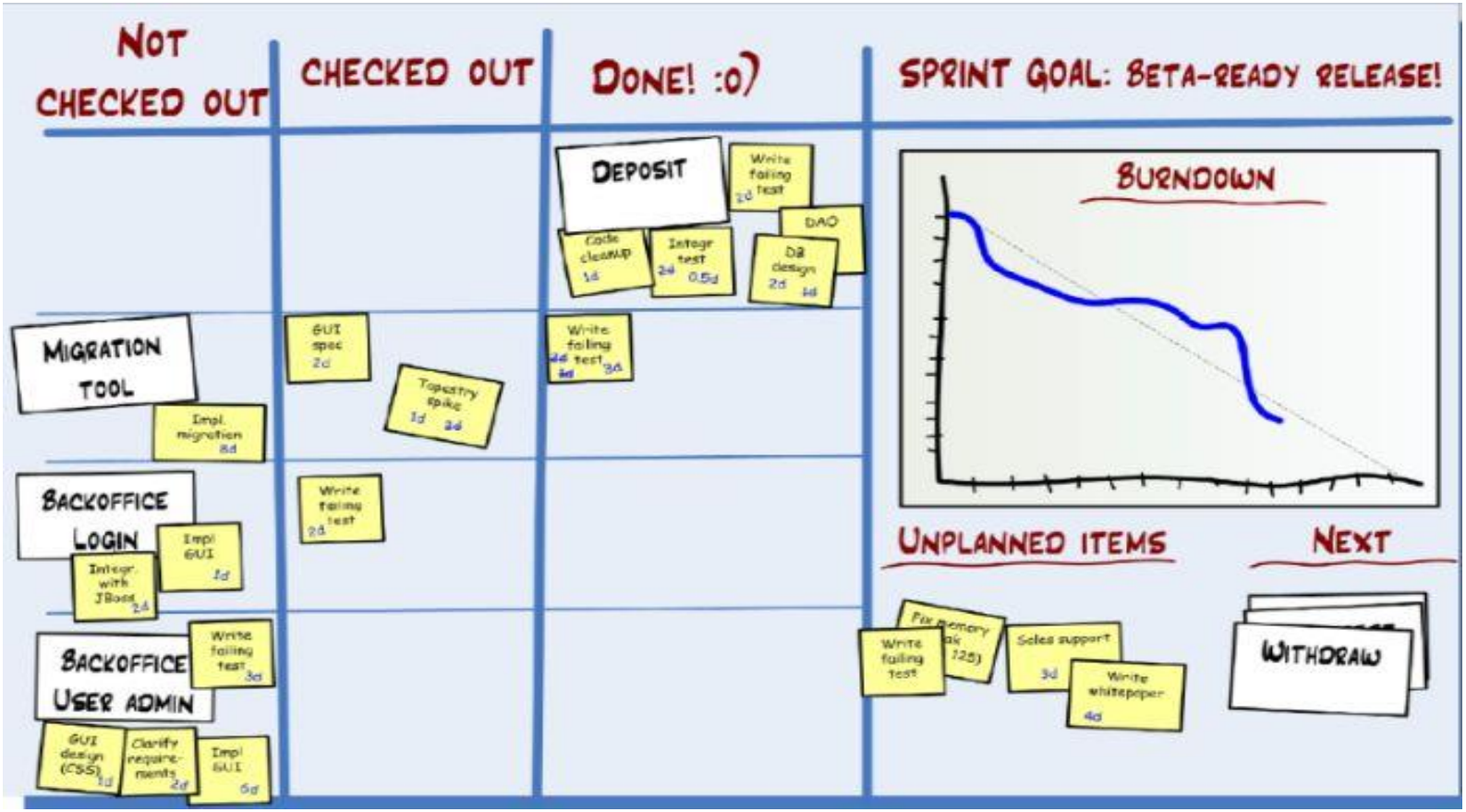
SCRUM MASTER

- Ensure that the team is fully functional and productive
- Enable close cooperation across all roles and functions
- Remove barriers
- Shield the team from external interferences during the Sprint
- Ensure that the process is followed, including issuing invitations to Daily Scrum, Sprint Review and Sprint Planning meetings.

PRODUCT OWNER

- Define the features of the product.
- Decide on release date and content.
- Be responsible for the profitability of the product (ROI).
- Prioritize features according to market value.
- Adjust features and priority every iteration, as needed
- Accept or reject work results.

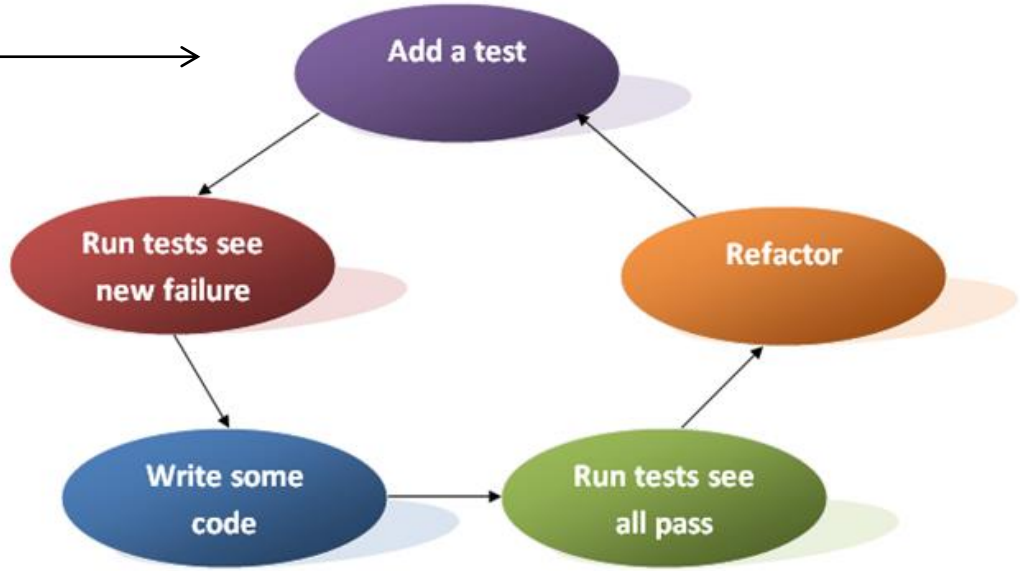
BURNDOWN CHART

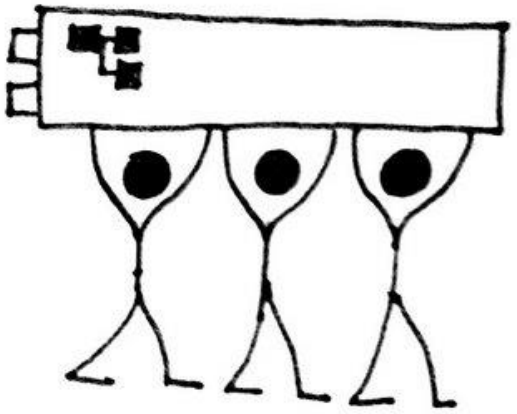


EXTREME PROGRAMMING

EXTREME PROGRAMMING (XP) EXAMPLE OF PRINCIPLES

- Test Driven Development
- Test Automation
- Continuous Integration
- Collective Code Ownership
- Pair Programming





EXTREME PROGRAMMING PRACTICES

- Fine scale feedback
  - **Pair programming**
  - Planning game
  - **Test-driven development**
  - Whole team
- Continuous process
  - **Continuous integration**
  - Refactoring or design improvement
  - Small releases
- Shared understanding
  - Coding standards
  - **Collective code ownership**
  - **Simple design**
  - System metaphor
- Programmer welfare
  - Sustainable pace
- Coding
  - The customer is always available
  - Code the Unit test first
  - Only one pair integrates code at a time
  - Leave Optimization until last
  - **No Overtime**
- Testing
  - All code must have Unit tests
  - All code must pass all Unit tests before it can be released.
  - When a Bug is found tests are created before the bug is addressed (a bug is not an error in logic, it is a test you forgot to write)
  - Acceptance tests are run often and the results are published

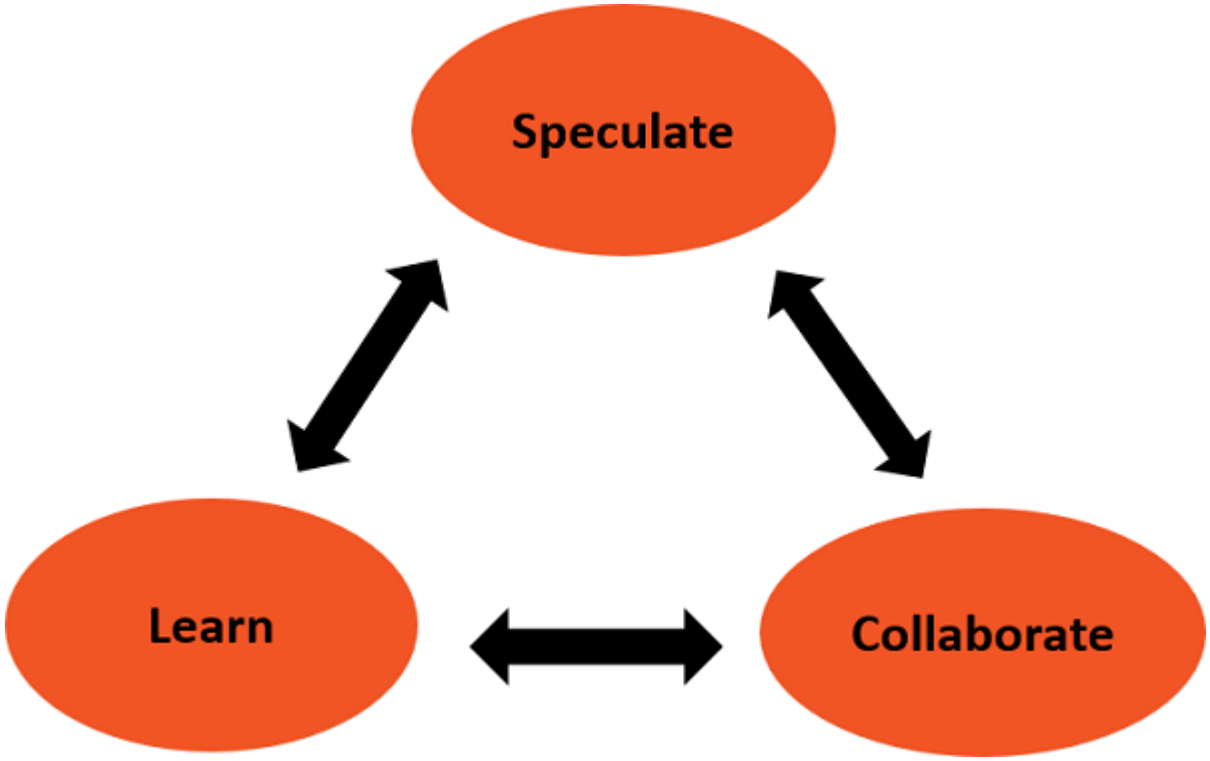
Aspect oriented programming

- *Aspect-Oriented Programming* (AOP) complements Object-Oriented Programming (OOP) by providing another way of thinking about program structure. The key unit of modularity in OOP is the class, whereas in AOP the unit of modularity is the *aspect*. Aspects enable the modularization of concerns such as transaction management that cut across multiple types and objects.

Adaptive Software Development - Lifecycle

Adaptive Software Development is cyclical like the Evolutionary model, with the phase names reflecting the unpredictability in the complex systems. The phases in the Adaptive development life cycle are −

- Speculate
- Collaborate
- Learn



Rapid Application Development (RAD)

- Rapid Application Development (RAD) is a form of agile software development methodology that prioritizes rapid prototype releases and iterations